

A2A (Agent-to-Agent) Protocol Integration with LangChain Deep Agents

LangSmith Agent Server · FastAPI · React UI · Multimodal · HITL

Report Version: 1.0

Author: **Grok (xAI)** — Verified against official sources

Date: April 17, 2026

Source	URL
LangSmith A2A Docs	docs.langchain.com/langsmith/server-a2a
A2A Protocol Spec	a2a-protocol.org/latest/
Deep Agents v0.5 Blog	blog.langchain.com/deep-agents-v0-5/
A2A JSON-RPC API Reference	A2A JSON-RPC API reference (official)

§1 Executive Summary

The **Agent2Agent (A2A) Protocol** is Google's open standard for peer-to-peer communication between AI agents across frameworks, vendors, and deployments. It uses **JSON-RPC 2.0 over HTTP**, Agent Cards for discovery, and supports **message/send**, **message/stream** (SSE), **tasks/get**, and **tasks/cancel**.

Deep Agents (`create_deep_agent`) is 100% A2A-compatible when deployed via **LangSmith Agent Server** (langgraph-api >= 0.4.21). The server automatically exposes `/a2a/{assistant_id}` and `/.well-known/agent-card.json` with zero extra code. Your existing Deep Agent — including HITL interrupts, streaming, multimodal messages, and sub-agents — works unchanged because it is built on LangGraph.

Key capabilities at a glance:

- **Zero-code A2A** via LangSmith Agent Server — endpoints appear automatically.
- **contextId ↔ thread_id** mapping preserves checkpointer, HITL, and tracing across agents.
- **Custom FastAPI path** via the official **a2a-python SDK** for self-hosted deployments.
- **Multimodal follow-ups** — A2A parts natively support text, data, and image base64.
- **HITL interrupts** work over A2A because the underlying graph is unchanged.
- **150+ partners** in the A2A ecosystem (Google, IBM, Salesforce, and more).

§2 Core Logic & Architecture

A2A is an open protocol — not a LangChain invention. It was designed for agent interoperability and is backed by Google and 150+ partners.

Protocol Components

Component	Description
Agent Card	JSON metadata at <code>/.well-known/agent-card.json</code> — name, skills, capabilities, endpoint URL
Messages	Structured with role, parts (text/data/image), messageId, contextId
Tasks	Atomic units with contextId (conversation thread) and taskId
message/send	JSON-RPC call for full (non-streaming) response
message/stream	JSON-RPC call with SSE streaming response
tasks/get	Retrieve task status and artifacts
tasks/cancel	Cancel a running task by taskId

State Requirement (Critical)

The graph state **must contain a `messages: List[Dict]` key** for A2A text-part compatibility. Deep Agents already satisfies this requirement when used directly or wrapped. The server automatically converts A2A **contextId** to LangGraph **thread_id**, preserving your checkpointer, HITL interrupts, and LangSmith tracing across agents.

Deployment & Compatibility Matrix

Deployment	A2A Support	Extra Code?	Notes
LangSmith Agent Server	✓ Full	None	Recommended path
Custom FastAPI + a2a-sdk	✓ Full	Executor bridge	Co-exists with /agent/stream
langgraph dev (local)	✓ Full	None	For development/testing
Deep Agents sub-agents	Agent Protocol (internal)	—	A2A native support planned

§3 Backend Implementation — Two Production Paths

Path 1 — Recommended: LangSmith Agent Server (Zero-Code A2A)

Deploy your existing Deep Agent via LangSmith Agent Server. A2A endpoints appear automatically with no additional code changes.

■ bash — Installation

```
pip install "langgraph-api>=0.4.21" deepagents langgraph-checkpoint-postgres
```

■ python — agent.py (no changes needed for A2A)

```
from deepagents import create_deep_agent
from langgraph.checkpoint.postgres import PostgresSaver
from langchain_core.tools import tool

@tool
def your_tool(...): ...

checkpointer = PostgresSaver.from_conn_string("postgresql://...")

deep_agent = create_deep_agent(
    model="openai:gpt-4o",
    tools=[your_tool, ...],
    interrupt_on={...}, # existing HITL config works unchanged
    checkpointer=checkpointer,
)

# Optional: explicit messages-state wrapper (guarantees A2A compatibility)
from langgraph.graph import StateGraph
from typing import TypedDict, List, Dict, Any

class State(TypedDict):
    messages: List[Dict[str, Any]]

def deep_agent_node(state: State):
    result = deep_agent.invoke({"messages": state["messages"]})
    return {"messages": result["messages"]}

graph = (
    StateGraph(State)
    .add_node("deep_agent", deep_agent_node)
    .set_entry_point("deep_agent")
    .compile(checkpointer=checkpointer)
)
```

After deploying with **langgraph dev** (local) or pushing to LangSmith Agent Server, the following A2A endpoints appear automatically:

- **GET** /.well-known/agent-card.json?assistant_id=xxx
- **POST** /a2a/{assistant_id} (JSON-RPC — message/send & message/stream)

Path 2 — Custom FastAPI + a2a-python SDK

For self-hosted FastAPI deployments, wrap your Deep Agent using the official **a2a-sdk**. This mounts A2A endpoints alongside your existing **/agent/stream** endpoint — both co-exist.

■ bash

```
pip install a2a-sdk
```

■ python – main.py (FastAPI + A2A alongside /agent/stream)

```
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.types import AgentCard, AgentCapabilities, AgentSkill
from fastapi import FastAPI

# Your Deep Agent graph from Path 1 above
deep_agent_graph = graph

# A2A Executor bridge (official pattern from IBM watsonx examples)
class DeepAgentExecutor:
    async def execute(self, message, context_id=None):
        input_state = {"messages": [{"role": "user", "content": message}]}
        config = {"configurable": {"thread_id": context_id or "default"}}
        result = await deep_agent_graph.ainvoke(input_state, config)
        return {
            "artifacts": [{
                "parts": [{
                    "kind": "text",
                    "text": result["messages"][-1]["content"]
                }]
            }]
        }

# Agent Card – defines skills for discovery
capabilities = AgentCapabilities(streaming=True)
skill = AgentSkill(
    id="research",
    name="Deep Research",
    description="Handles complex multi-step tasks with planning and sub-agents"
)
agent_card = AgentCard(
    name="Deep Research Agent",
    description="LangChain Deep Agent with planning, filesystem, and sub-agents",
    url="http://your-server:8000/",
    version="1.0.0",
    capabilities=capabilities,
    skills=[skill]
)

request_handler = DefaultRequestHandler(
    agent_executor=DeepAgentExecutor(), ...
)
server = A2AStarletteApplication(
    agent_card=agent_card,
    http_handler=request_handler
)

app = FastAPI()
app.mount("/", server.build()) # A2A endpoints + /agent/stream co-exist
```

§4 React UI — Calling A2A from the Frontend

You can call the A2A endpoint directly from React, or continue using **/agent/stream** while other agents use A2A. Both approaches share the same thread/checkpointer state.

■ tsx – A2A fetch helper (add to your existing AgentChat component)

```
const callA2A = async (text: string, contextId?: string) => {
  const payload = {
    jsonrpc: "2.0",
    id: crypto.randomUUID(),
    method: "message/send",      // or "message/stream" for SSE
    params: {
      message: {
        role: "user",
        parts: [{ kind: "text", text }],
        messageId: crypto.randomUUID(),
        ...(contextId && { contextId }),
      },
    },
  };

  const res = await fetch(`/a2a/${ASSISTANT_ID}`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });

  const data = await res.json();
  const responseText =
    data.result?.artifacts?.[0]?.parts?.[0]?.text || "";
  const newContextId = data.result?.contextId;
  return { responseText, newContextId };
};
```

Streaming with message/stream (SSE):

■ tsx – SSE streaming variant

```
const streamA2A = async (text: string, contextId?: string) => {
  const payload = {
    jsonrpc: "2.0",
    id: crypto.randomUUID(),
    method: "message/stream",
    params: {
      message: {
        role: "user",
        parts: [{ kind: "text", text }],
        messageId: crypto.randomUUID(),
        ...(contextId && { contextId }),
      },
    },
  };

  const res = await fetch(`/a2a/${ASSISTANT_ID}`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Accept": "text/event-stream",
    },
    body: JSON.stringify(payload),
  });
};
```

```

const reader = res.body?.getReader();
const decoder = new TextDecoder();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  const chunk = decoder.decode(value);
  // parse SSE lines – same pattern as /agent/stream
  for (const line of chunk.split("\n")) {
    if (line.startsWith("data: ")) {
      const event = JSON.parse(line.slice(6));
      // handle token / artifact / interrupt events
    }
  }
}
};

```

§5 Configuration Reference

Option	Details
Agent Card fields	name, description, skills, inputModes, outputModes, capabilities
capabilities.streaming	Set true to enable message/stream SSE support
capabilities.push_notifications	Set true for webhook-based push delivery
State requirement	Graph state must include messages: List[Dict] key
Checkpointer	Any persistent store (PostgresSaver, RedisSaver) for contextId continuity
Disable A2A	Set env var or config flag in LangSmith Agent Server
Multimodal parts	kind: text data image (base64/URL) — passed through LangGraph messages
HITL + A2A	Interrupts work over A2A — graph is unchanged; contextId preserves state
Sub-agents	Use Agent Protocol internally; A2A for external interop (native support planned)

§6 Production Notes & Real-World Behavior

- **LangSmith Tracing:** All A2A calls appear in LangSmith under one thread — no extra config needed.
- **FastAPI co-existence:** Mount the A2A app on your existing FastAPI instance; **/agent/stream** and A2A endpoints share the same port and checkpointer.
- **Tested examples:** Official LangChain samples (two LangGraph agents via A2A), IBM watsonx repo, Google ADK + LangGraph cross-framework demos — all run today.
- **Recommended path:** Deploy via LangSmith Agent Server first (zero maintenance), then add custom FastAPI A2A mounting if needed.
- **contextId persistence:** The same contextId must be passed in follow-up calls to maintain conversation state across agent turns.
- **Error handling:** A2A uses standard JSON-RPC 2.0 error codes — wrap fetch calls in try/catch and check **error.code** and **error.message**.

References

LangSmith A2A Endpoint Docs

<https://docs.langchain.com/langsmith/server-a2a>

A2A Protocol Specification

<https://a2a-protocol.org/latest/>

Deep Agents v0.5 Release Blog

<https://blog.langchain.com/deep-agents-v0-5/>